

MagicCode Super-Resolution Software Integration Guide

Document Version: v2.2.0. Release Date: September 30, 2026.

1. Libraries

Platform	Archive	Notes
Android	lib/android/libmagic_sr.a	NEON CPU; Vulkan / OpenGL ES GPU; temporal GPU yes
iOS	lib/ios/libmagic_sr.a	NEON CPU; Metal GPU; temporal GPU yes
macOS arm	lib/mac_arm/libmagic_sr.a	NEON CPU; Metal GPU; temporal GPU yes
Windows	lib/windows/libmagic_sr.lib	x86 CPU; Vulkan / Direct3D 11 / OpenGL 4.3 GPU; temporal GPU yes
Linux x86_64	lib/linux/libmagic_sr.a	x86_64 CPU spatial (AVX2); no GPU backend objects; Vulkan temporal via source build

v2.2.0 ships one archive per platform; the v2.1.0 Enable archive (libmagic_sr_enable.a / libmagic_enable_sr.lib) and mc_enable.h are removed. All integrations link libmagic_sr and use mc_nscaler_*. Windows temporal objects are inside libmagic_sr.lib; do not also link magic_sr_win_vk / magic_vulkan_temporal / magic_vulkan_taa. Linux: lib/linux/libmagic_sr.a (CPU spatial only); rebuild with tools/build_linux_lib.sh and link -lm -lpthread.

2. Spatial integration (nscaler)

1. Include mc_interface.h and link libmagic_sr.
2. Set ctrl_param_t.model_path and create with mc_nscaler_control(MC_NSCLER_CMD_SET_PARAM), or pass a NULL handle to mc_nscaler_enable for a default 2x SPATIAL_SPEED create.
3. Bind per-frame width/height on mc_nscaler_input_frame_t; the first enable sets the handle size.
4. Call mc_nscaler_enable(&handle, &in_frame, &out_frame) each frame; release only with mc_nscaler_disable.

3. Temporal session integration

5. Include mc_interface.h and link libmagic_sr.
6. Fill ctrl_param_t (temporal alg_mode, scaler in (1, 8], gpu_context, create-stage depth/HDR). ctrl_param_t has no struct_size/width/height; the size comes from the first enable frame.

7. Create with `mc_nscaler_control(MC_NSCALER_CMD_SET_PARAM)`. Vulkan/D3D11 device import is required as documented in `mc_interface.h`.
8. Create once and keep the handle across frames. Each frame: zero-init `temporal_frame_t`, set `struct_size`, bind depth/motion (and optional masks), set jitter/frame_index/camera, attach to `mc_nscaler_input_frame_t.frame`, bind color/output on `mc_nscaler_input_frame_t / mc_nscaler_output_frame_t` (including Vulkan layouts on color/depth/motion/output), set `command_buffer` on Vulkan.
9. `mc_nscaler_enable` only records or dispatches for that frame. On Vulkan it does not submit or wait. Set `reset_history` on load/teleport.
10. After the caller has submitted and waited on recorded GPU work, `mc_nscaler_disable` in shutdown. Do not disable immediately after `mc_nscaler_enable` while GPU work may still be in flight.

Temporal Super-Resolution

Temporal super-resolution runs through the nscaler session API: `libmagic_sr.a` (Windows: `libmagic_sr.lib`) + `mc_interface.h` + `mc_nscaler_enable / mc_nscaler_control / mc_nscaler_disable`. v2.2.0 replaced the v2.0.0 `MC_Init / MC_Process / MC_Uninit` entry and removed the v2.1.0 `MC_Enable` helper (`mc_enable.h` is deleted). A NULL *handle on `mc_nscaler_enable` creates a session with defaults (2x SPATIAL_SPEED) and processes the first frame.

Modes

Value	Enumerator	Role
0	SPATIAL_SPEED_MODE	Spatial, throughput first
1	SPATIAL_BALANCED_MODE	Spatial, quality/cost trade-off
2	TEMPORAL_SPEED_MODE	Temporal, throughput first
3	TEMPORAL_BALANCED_MODE	Temporal, canonical FSR-family path

- Temporal `scaler_factor` is (1, 8]; 1.0 itself is rejected. Spatial remains [1.0, 8.0].
- `TEMPORAL_SPEED_MODE` on Metal, Vulkan, and OpenGLS currently uses a lightweight Convert → Activate → Upscale pipeline. Direct3D 11 and desktop OpenGL 4.3 Speed use the FSR-family temporal path (same family as Balanced, not a separate customer API).
- `TEMPORAL_BALANCED_MODE` uses the canonical FSR-family path on every GPU temporal backend. Desktop Balanced edge-resolve is supported on Windows Vulkan, Windows Direct3D 11, desktop OpenGL 4.3, and Apple Metal. Android Vulkan and Android OpenGLS do not run edge-resolve. If `enable_sharpening` is on, RCAS runs after edge-resolve.
- CPU backends (X86 / NEON) cannot run temporal modes.
- The API accepts (1, 8]. Exact 2x has optimized kernels. Not every scale or device is claimed as device-run.
- Internal pipeline names are implementation detail, not public configuration.

Minimum per-frame inputs

- `mc_nscaler_input_frame_t` / `mc_nscaler_output_frame_t`: caller-owned color and output GPU resources.
- `mc_nscaler_input_frame_t.frame`: non-NULL `temporal_frame_t*` with `struct_size = sizeof(temporal_frame_t)`.
- `temporal_frame_t.depth` and motion at input resolution.
- `jitter_offset_x/y` in input pixels, top-left, +X right, +Y down.
- `frame_index` / `reset_history`; `camera_near` / `camera_far` / `camera_fov_y` (radians).
- `gpu_context` at create (`mc_nscaler_control SET_PARAM`, or the first `mc_nscaler_enable`): Vulkan requires `physical_device + device`; D3D11 requires `device`; Metal device may be NULL; GL/GLES use the current context (library never makes current).
- Vulkan temporal requires `mc_nscaler_input_frame_t.command_buffer` to be a recording `VkCommandBuffer`. Metal `command_buffer` is optional. D3D11 / GL / GLES leave it NULL.
- Create-stage `depth_reversed` / `depth_infinite` / `hdr_color` on `ctrl_param_t` (all-zero = conventional-Z, finite far, LDR). Immutable for the handle lifetime.

Motion, depth, and masks

- Motion is current → previous. The library never negates MVs.
- `motion_vector_scale_x/y` is the only MV numeric conversion. (0,0) defaults to (input_width, input_height). Mixed zero is rejected.
- Depth is GPU device depth in [0, 1]. Linear view-space depth is not accepted.
- `reactive` / `transparency` are optional. Explicit caller textures win per channel.
- When omitted, the library derives an approximate mask. That is not a semantic transparency mask.
- Speed on Metal/Vulkan/GLES may derive both missing `reactive` and an approximate transparency. FSR-family paths (Balanced on all backends; Speed on D3D11/desktop GL) derive missing `reactive` only and bind an internal zero transparency texture.
- Both temporal modes enable depth+MV history rejection by default.
- Diagnostic only: `MAGICSR_TAAU_AUTO_MASK=0` and `MAGICSR_TAAU_HIST_REJECT=0` can turn those defaults off. They are not a customer configuration path.

4. Unity / Unreal

Plugins wrap Enable for spatial upscaling. They do not provide per-frame depth/motion/jitter. Full temporal belongs on the Native session API. See `plugin/unity/USAGE_GUIDE.md` and `plugin/ue/USAGE_GUIDE.md`.

5. Vulkan command-buffer notes

- `command_buffer` must already be recording. MagicSR does not begin, end, submit, or wait.
- layout on color, depth, motion, and `image_out` must be the actual layout at entry (not 0 / UNDEFINED).

- Keep the handle across frames. `mc_nscaler_enable` only records. Caller submit/wait, then `mc_nscaler_disable` at shutdown.
- `physical_device`, `device`, and images must be the same `VkDevice`.